

2024년 이브와 ICT멘토링 개 발 보 고 서

2024. 9.

프로젝트명

NLP 기반 소셜 IP 확장 솔루션

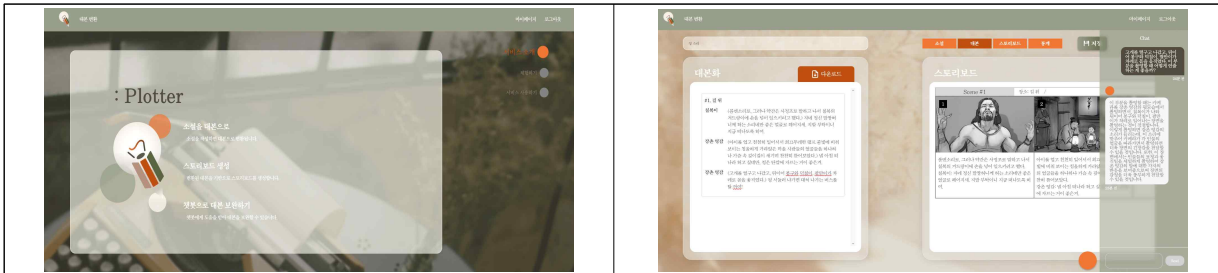
요 약 본

프로젝트 정보	
프로젝트명	NLP 기반 소설 IP 확장 솔루션
주제 영역	☐ 생활 ☐ 업무 ☐ 공공/교통 ☐ 금융/핀테크 ☐ 의료 ☐ 교육 ☐ 유통/쇼핑 ☒ 엔터테인먼트
기술 분야	☒ SW·AI ☐ 방송·콘텐츠 ☐ 블록체인·융합 ☐ 디바이스 ☐ 차세대보안 ☐ 미래통신·전파
성과 목표	☒ 논문게재 및 포스터 발표 ☐ 앱등록 ☐ 프로그램등록 ☐ 특허 ☐ 기술이전 ☒ 실용화 ☒ 공모전(ICT멘토링 이브와 공모전) ☐ 기타()
프로젝트 소개	본 프로젝트는 사용자가 작성한 소설을 대본으로 변환하고 변환된 대본으로 스토리 보드를 생성하는 알고리즘을 통해 제작 과정을 자동화하여 각색 과정에서 소요되는 자원을 최소화하며, 동시에 드라마, 영화, 연극 등의 시청각 창작물 제작을 위한 토대를 마련한다.
개발 배경 및 필요성	2023년 영상물로 재탄생한 작품이 네이버웹툰에서 30개 이상, 카카오편터엔터테인먼트에서 50개 이상이다. 웹툰과 소설을 원작으로 한 드라마와 영화가 인기를 끄는 이유는 신선한 소재와 검증된 IP 덕분이다. 이 프로젝트는 창작물의 영상화 과정 중 대본화와 스토리보드 생성을 자동화하여 시간과 인건비를 크게 줄이고 광고, 애니메이션, 영화, 뮤지컬 제작 등에 도움이 될 것이라 예상한다. 특히, 대본과 스토리보드가 원작 소설의 내용을 충실히 반영하는 알고리즘을 통해 생성되므로 원작과의 싱크로율을 중시하는 팬층의 기대를 충족시킬 수 있다.
프로젝트 특·장점	- 소설-대본-스토리보드 생성 알고리즘 자체 개발 - 사용자 창작 맞춤형 서비스를 제공함으로써 창작물 제작 효율성 극대화 - 사용자가 작성한 소설의 미디어 프랜차이즈화: 다양한 시청각 작품으로의 확장성
주요 기능	- 소설을 대본 형식으로 변환 - 소설, 대본 기반으로 스토리보드 생성 - 작품 통계 시각화 - 소설 기획 및 각색 도움 가이드 챗봇
기대효과 및 활용 분야	- 소설의 극본화, 스토리보드 생성 자동화: 연극, 영화 등 IP 확장, 콘텐츠 융합 촉진 - 독자와 새로운 관객층에 실감나는 경험을 제공 - 챗봇을 통해 향상된 검색 기능 제공, 창작 과정의 일관성 유지 - 교육자료 개발, 전통 이야기의 현대적 재해석 등으로 상업적, 교육적 가치를 높이며 문화산업 확장에 기여

본 문

I. 프로젝트 개요

1. 프로젝트 소개



1) 기획 의도

- 사용자가 작성한 소설을 대본으로 변환하고 변환된 대본으로 스토리보드를 생성하는 알고리즘을 통해 제작 과정 자동화, 각색 과정에서 드는 시간과 인건비 최소화.
- 드라마, 영화, 연극 등의 시청각 창작물을 제작할 수 있도록 토대 마련

2) 프로젝트 내용

소설의 대본화: 사용자가 작성한 소설에서 중심 등장인물 추출, 대사-비대사 분리, 발화자-대사 매칭 등의 자연어 처리 알고리즘을 거쳐 대본으로 변환.

대본 기반 스토리보드 생성

- 대본에서 드러나는 장소를 기준으로 씬(Scene)을 나누고 인물의 행위를 기준으로 컷(Cut)을 나눔.
- 등장인물의 행위를 이미지 생성 모델의 입력값으로 넣어 컷 단위로 이미지 생성. 장소와 사용되는 소품, 액션과 대사가 배치된 스토리보드 생성.

작품 정보 시각화: 장소, 등장인물 등의 작품 구성 정보와 등장인물 출연율을 마인드맵, 바 차트로 시각화하여 사용자가 작품 정보를 한 눈에 확인.

작품 각색 가이드 챗봇

- 사용자가 작성한 소설을 바탕으로 사용자의 질의에 답변을 제공, 소설-대본-스토리보드 생성 과정에서 스토리의 일관성을 유지할 수 있도록 도움.
- 특정 장면에 대해 연출과 카메라 앵글에 대한 조언을 주는 등 각색 및 촬영 단계에서 필요한 아이디어 제공.

2. 개발 배경 및 필요성

1) 개발 배경

최근에는 소설, 웹툰을 기반으로 영화, 애니메이션으로 재탄생하는 사례가 증가

하고 있는데, 신선한 소재와 스토리라인, 검증된 지식재산권(IP)이 가장 큰 이유이다. 네이버웹툰에 따르면 2023년에 30개 이상의 콘텐츠 IP가 드라마 등의 영상으로 재탄생했으며 카카오엔터테인먼트의 콘텐츠 IP도 50개 이상 판매되었다.

영상물을 제작하는 과정에서 스토리보드는 필수적이다. 스토리보드는 영화, 드라마, 광고 등의 영상물을 제작하기 위해 반드시 작성해야 하는 문서로, 대본을 바탕으로 지문, 대사, 장소, 소품 등 장면에서 필요한 모든 내용을 기록한다. 각 색은 원작을 기반으로 원작자, 대본가가 협업하여 대본 작업을 완료한 후, 평균 3주에서 8주 동안 촬영 감독, 원작자, 아티스트가 스토리보드 작업을 진행하며 이 과정에서 많은 시간과 인력이 든다.

콘텐츠의 영상화가 활발히 이루어지는 가운데, 본 서비스는 사용자의 창작물을 바탕으로 대본과 스토리보드를 자동으로 생성한다. 이를 통해 인건비와 제작 시간을 줄임으로써 작품의 경쟁력을 확보할 수 있을 것이다.

2) 필요성

- 본 서비스는 단순한 요약, 분류를 넘어 텍스트 창작물의 영상화를 목적으로 진행하므로 광고 콘티 제작, 애니메이션 제작, 영화/드라마 제작, 뮤지컬 및 연극의 무대 제작에도 큰 도움이 될 것으로 예상된다.
- 오픈서베이 웹툰 트렌드 리포트에 따르면, 웹툰을 영상화할 경우 75.3%의 웹툰 독자들은 ‘원작 스토리의 내용’과의 싱크로율을 중요하게 생각한다고 답했다. 기존 대본과 스토리보드 작업에서 발생할 수 있는 일관성 부족 문제를 해결하여, 원작 소설을 기반으로 생성된 대본과 스토리보드는 스토리의 일관성을 유지하는 점에서 독자의 기대감을 향상시킬 것이다.

3. 프로젝트 특·장점

1) 소설-대본-스토리보드 생성 알고리즘 자체 개발

- 등장인물 선정, 등장인물의 행위 추출, 등장인물과 대사 매칭 알고리즘을 구현하여 소설 텍스트를 대본으로 변환하고 스토리보드를 생성하는 자체 개발 알고리즘이다.

2) 사용자 창작 맞춤형 서비스를 제공함으로써 창작물 제작 효율성 극대화

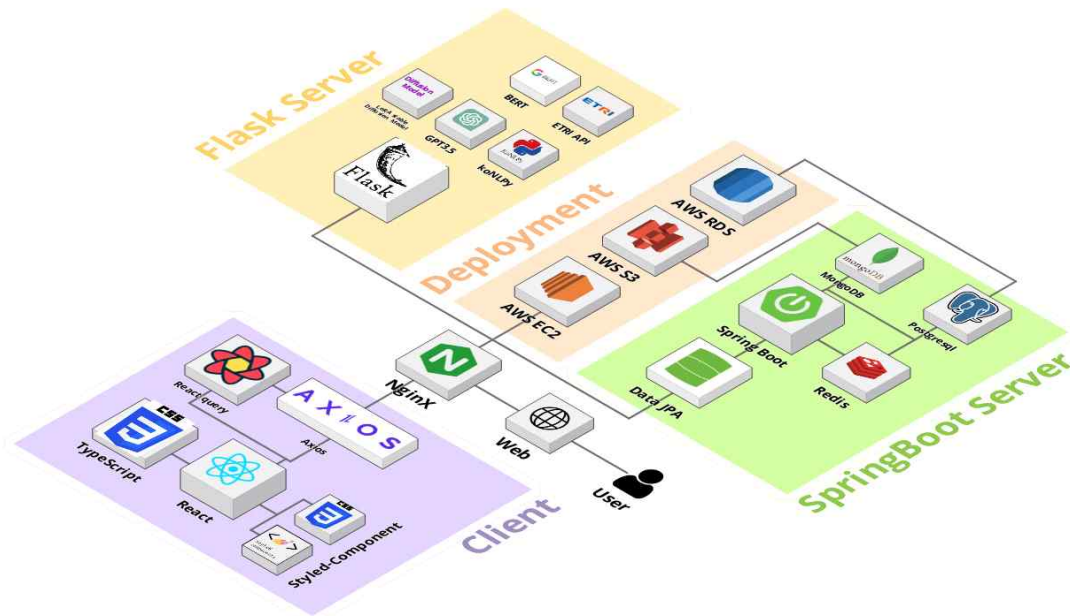
- 대본과 스토리보드를 바탕으로 사용자의 질문에 대한 답변을 제공하는 ‘창작물 맞춤형 챗봇’을 통해 스토리의 일관성을 유지하여 제작 시간과 인건비 절감의 효과를 가져온다.
- 특정 장면에서의 카메라 앵글과 배경과 분위기 연출에 대한 답변을 통해 각색 및 촬영 단계에서 창작물에 대한 조언을 구할 수 있다.

3) 사용자가 작성한 소설의 미디어 프랜차이즈화: 다양한 시청각 작품으로의 확장성

- 사용자가 작성한 소설을 바탕으로 스토리보드를 스스로 생성하여 소설을 드라마, 영화, 연극, 애니메이션 등의 시청각 창작물로 확장할 기회를 제공한다.

II. 프로젝트 내용

1. 프로젝트 구성도



1) React 서버 단

- 프로젝트 환경: React, Typescript
- 컴포넌트 디자인: Styled-component, css
- API 통신: React query, Axios
- 데이터 관리: Context API

2) Spring boot 서버 단

- 프로젝트 환경: Spring Boot
- API 설계: Data JPA
- DB 및 캐싱: Postgresql, MongoDB, Redis

3) Flask 서버 단

- 프로젝트 환경: Flask
- 소설의 대분화: koNLPy, ETRI API, Spacy-ko, mecab, RoBERTa
- 스토리보드 생성: LoRA Diffusion Model SDXL
- 챗봇: GPT3.5-turbo-instruct

4) 배포

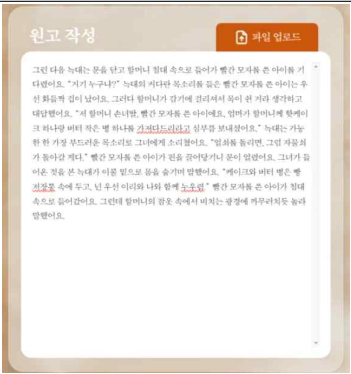
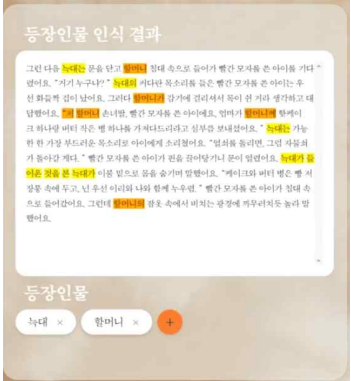
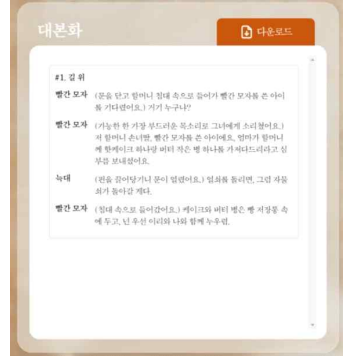
- Nginx: React 서버, Spring Boot 서버 프로젝트 배포
- AWS EC2: 클라우드 환경에 프로젝트 배포
- AWS S3: 메타데이터 관리 및 저장
- AWS RDS: 클라우드 환경에서 관계형 데이터베이스 운영 및 배포
- Vast.ai: Flask Server 배포

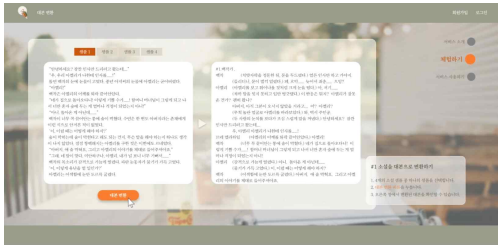
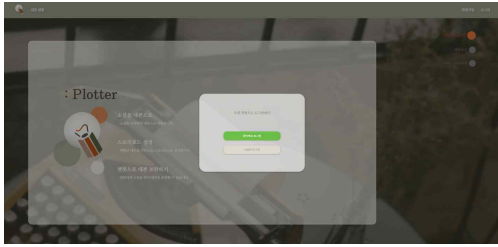
2. 프로젝트 기능

1) 전체 기능 목록

구분	기능	설명	현재진척도(%)
S/W	소설 속 등장인물 추출	상호참조해결 알고리즘을 통해 소설 대본화 과정에 필요한 등장인물 목록을 추출한다.	100
	소설을 대본 형식으로 변환	입력된 소설 데이터에서 등장인물의 발화와 행위를 인물에 따라 구분하여 대본 형식으로 변환한다.	100
	스토리보드 생성	소설에서 추출한 장소, 등장인물의 액션, 변환한 대본을 바탕으로 컷(Cut) 단위의 프롬프트로 가공하여 text to image 모델을 통해 생성한 예상 이미지를 보여준다. 등장인물의 액션, 대사, 장소, 소품을 삽입한 스토리보드 형식으로 웹페이지에 띄운다.	100
	작품 정보 시각화	소설 내용에서 추출한 등장인물, 장소 등의 정보와 등장인물의 출연 비율을 마인드맵과 차트로 시각화한다.	100
	사용자 창작물의 기획 가이드 챗봇	소설 및 대본 데이터를 토대로 만들어진 GPT-3.5-turbo-instruct(구매 S/W) 챗봇을 통해 사용자의 질문에 따른 정보를 제공하여 스토리의 일관성을 유지한다. 또한, 씬과 컷 단위로 스토리의 전개와 구성을 조정할 때 챗봇으로부터 아이디어를 얻을 수 있어 사용자의 창작 과정에 기여한다.	100
	서비스 데모	서비스를 본격적으로 사용하기 전, 사용자가 소설 대본화 기능을 체험할 수 있다. 주어진 예시 4가지의 대본 변환 결과를 확인할 수 있다.	100
	회원 관리	사용자는 회원가입을 통해 서비스를 이용할 수 있다. 기존에 보유 중인 Google과 Naver 계정을 연결해 본 서비스를 이용할 계정을 생성할 수 있다.	100
	변환 내역 목록 확인	사용자는 대본 및 스토리보드 변환 및 생성 목록을 마이페이지에서 확인할 수 있다.	100
H/W	-	-	-

2) S/W 주요 기능

기능	설명	프로젝트 실물사진
변환 페이지: 소설 텍스트 입력	변환 대상이 되는 소설 원고를 입력하는 단계. 사용자는 화면에 텍스트를 직접 입력하거나, txt 파일을 업로드할 수 있다.	
변환 페이지: 등장인물 확인	대본화 과정 중, 소설 내의 등장인물 인식 결과를 확인하는 단계. 사용자는 목록에서 등장인물을 추가하거나 삭제할 수 있다.	
변환 페이지: 소설 대본 형식 변환 결과 확인	소설의 대본화 결과물을 확인하는 단계. 사용자는 인물의 대사와 행위가 제대로 연결되었는지 확인하고, 대본을 수정할 수 있다.	
변환 페이지: 스토리보드 생성	소설과 대본을 토대로 생성된 스토리보드를 확인하는 단계. 컷 단위로 예상 이미지, 촬영 기법, 디렉션이 생성된다.	
변환 페이지: 작품 통계	소설에서 등장인물, 장소 등의 정보를 추출하여 마인드맵 형태의 그래프로 시각화한다.	

변환 페이지: 사용자 창작물의 맞춤형 기획 가이드 챗봇	사용자는 챗봇과의 질의응답을 통해 스토리의 일관성을 유지하고, 각색 및 촬영 단계 도움을 받을 수 있다. 챗봇은 사용자가 입력한 소설 데이터와 변환된 대본 데이터를 토대로 학습하며, 사용자가 질문형 채팅을 보내면 이에 답변하는 방식으로 동작한다.	
메인 페이지: 서비스 데모	서비스를 본격적으로 사용하기 전, 사용자는 본 서비스에서 제공하는 기능 중 하나인 '소설의 대본화 기능'을 체험할 수 있다. 주어진 4가지 예시 중 하나를 선택하여 체험하기 버튼을 누르면 대본 변환 결과를 확인할 수 있다. 비로그인 사용자도 체험할 수 있다.	
회원 관리	Plotter 서비스의 핵심 서비스인 소설의 대본화, 스토리보드 생성 및 챗봇 기능은 로그인 사용자만 이용할 수 있도록 구현했다. Google과 Naver 소셜 로그인을 통해 계정을 생성할 수 있다.	
마이페이지: 대본 목록 확인	마이페이지에서 사용자 정보(이메일), 사용자가 생성한 대본 및 스토리보드 목록을 확인할 수 있다.	

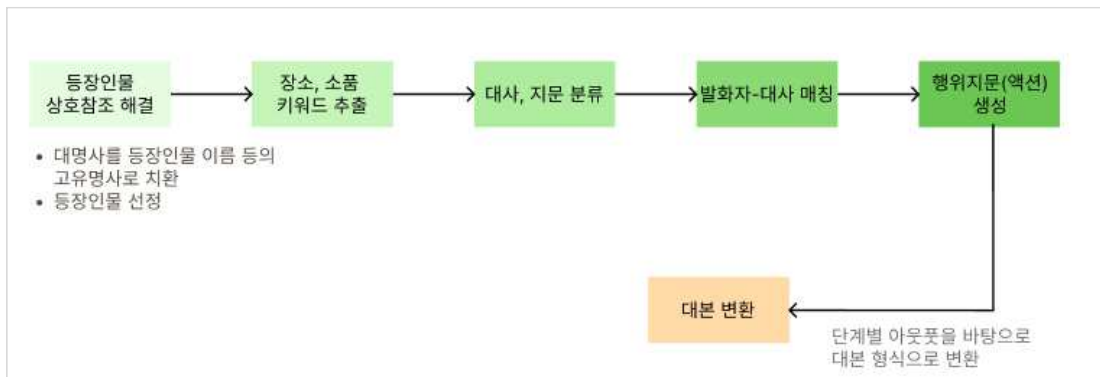
3) H/W 주요 기능

기능/부품	설명	프로젝트 실물사진
-	-	-

3. 주요 적용 기술

주요 적용 기술	사용 모델 및 라이브러리	기술 세부사항
1) 소설 데이터를 대본 형식으로 변환	BERT, kolnpy, ETRI API, Spacy-ko, RoBERTa	- 등장인물의 상호참조해결 및 중심 등장인물 선정 - 장소, 소품 키워드를 추출하는 spacy 커스텀 모델 학습 - 지문, 대사 분류 - 발화자 대사 추정 알고리즘 - 등장인물의 행위 지문 추출
2) LoRA stable diffusion model을 이용한 스토리보드 생성	LoRA Stable Diffusion	- 컷 단위의 대본 텍스트를 입력으로 넣어 상황에 맞는 이미지 생성 - 생성된 이미지와 대본 텍스트를 조합하여 스토리보드 생성
3) GPT-3.5를 이용한 사용자 창작물의 기획 가이드 챗봇	GPT 3.5 turbo instruction	- 사용자의 창작물에 대한 정보 및 아이디어 제공 기능 - 창작물의 일관성 유지
4) React와 Spring Boot를 사용한 웹서비스 구축 및 배포	React, Spring Boot AWS EC2, Vast.ai	-

1) 소설 데이터를 대본 형식으로 변환



〈그림〉 대본 변환 알고리즘 흐름도

텍스트 소설의 대본화 알고리즘은 소설 속 등장인물 탐색 및 선정, 장소·소품 키워드 추출, 대사·지문 분리, 발화자와 대사 매칭, 등장인물의 행위 지문 생성으로 총 5단계를 거친다. 이후 텍스트 처리를 통해 소설을 대본의 형식으로 변환하고 이를 텍스트 파일(.txt)로 저장한다.

1-1) 소설 속 등장인물 탐색 및 선정, 대명사를 등장인물 고유명사로 치환

사용자가 입력한 소설 텍스트에서 등장하는 인물을 파악하여 상호참조해결을 거쳤고, 대명사에 해당하는 명사구를 고유명사로 치환했다. 상호참조해결(Resolving cross-reference)이란 한 개체에 대해 동일한 개체를 의미하는 명사구

들을 찾아 그룹핑하는 작업이다. 예를 들어, ‘왕자’라는 등장인물은 ‘왕자님’, ‘그’, ‘남자’로 같은 인물을 지칭하지만 다양한 명사구로 표현되고 있으므로 3개의 명사구는 하나의 그룹으로 묶인다. 이는 ETRI에서 제공하는 상호참조해결 API를 이용하여 소설 속 명사에 대해 그룹화된 명사구들의 결과를 얻었으며, 품사 태깅과 개체명 인식을 통해 등장인물에 해당하는 그룹핑된 명사구를 추출했다.

등장인물별로 그룹핑된 명사구들 중에 등장인물을 대표하는 명사구(중심어)를 선정한다. 선정 기준은 ①소설에서 언급되었던 명사구의 개수가 가장 많은 것과 ②대명사가 아닌 명사구여야 한다. 만약 한 등장인물에 대해 언급된 명사구의 개수가 같을 경우, 소설에서 맨 처음에 언급된 명사구를 사용한다. 만약 한 등장인물에 대해 ‘왕자’, ‘왕자’, ‘그’, ‘남자’로 묶여 있을 때, 중심어는 언급된 명사구가 개수가 가장 많은 ‘왕자’가 된다. 또한, ‘왕자’, ‘왕자’, ‘남자’, ‘남자’로 묶여 있을 경우, ‘왕자’와 ‘남자’가 언급된 수는 같으므로, 소설에서 먼저 언급된 ‘왕자’를 중심어로 선정한다.

왕자: ['왕자', '그', '그는', '남자는', '남자', '왕자', '왕자', '왕자님'] → 중심어: '왕자' 공주: ['공주', '공주', '그녀는', '그녀', '그 여자는', '여자', '여자'] → 중심어: '공주'

<예시> 중심어 선정 방법: 그룹핑된 명사구 카운팅

대명사에 해당하는 명사구들의 문장 내 인덱스값을 통해 명사구의 위치를 찾고 중심어로 치환한다. konlpy 라이브러리를 통해 명사구를 형태소 단위로 분리하고 ①대명사일 경우, 해당 단어를 중심어로 치환한다. ②조사가 포함된 대명사인 경우, 대명사와 조사를 분리 후, 대명사를 중심어로 치환한 후 조사를 합친다. 본 서비스에서는 konlpy가 대명사에 해당하는 명사구를 일반명사로 인식하는 문제가 있어, 대명사 사전을 만들었고 인식에 문제가 있는 명사구는 사전에 추가하는 작업을 거쳤다.

중심어: 왕자 치환 전: 그는 공주를 바라보면서 말을 걸었습니다. 치환 후: 왕자 는 공주를 바라보면서 말을 걸었습니다.
--

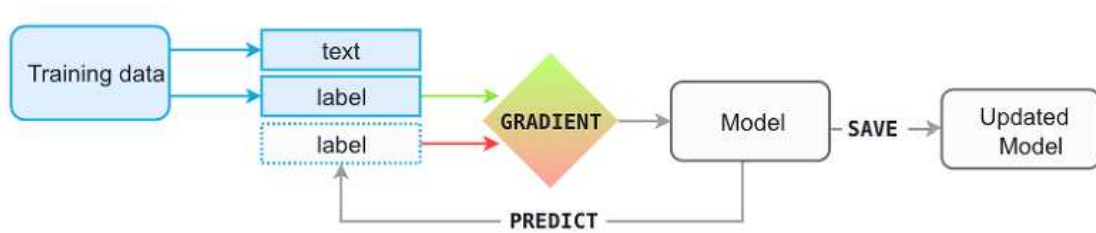
<예시> 대명사가 포함된 명사구를 중심어로 치환

1-2) 소설 속 장소, 소품 키워드를 추출하는 커스텀 모델 학습

Spacy 패키지 기반 모델을 학습하여 소설에서 등장하는 장소, 소품 키워드를 추출하는 커스텀 모델을 개발한다. Spacy는 자연어 처리를 위한 Python 기반의 오픈 소스 라이브러리로 개체명 인식(Named Entity recognition)을 지원한다.

Spacy를 활용한 훈련 과정은 여러 단계로 구성되며, 특히 Tok2Vec 모델과 전이

기반 파서(Transition-Based Parser)를 사용한 임베딩 및 라벨 예측 과정이 핵심이다.



〈그림〉 Spacy 훈련 과정

1-2-1) 데이터 준비

Spacy 모델을 학습시키기 위해, 소셜 텍스트 데이터를 직접 라벨링한 학습 데이터를 준비했다. 학습 데이터는 텍스트와 그에 대응하는 라벨(장소, 소품)로 구성된다. 라벨은 각 텍스트의 특정 부분이 장소나 소품과 같은 개체명에 해당 여부를 나타낸다.

```

1 data = [
2   ("옛날 옛적에요, 어떤 마을에요, 귀여운 사골 소녀가 살고 있었어요. 우쭐 대쁘고 귀여운 아이였답니다.", {"entities": [{"9,14,"location"}]}),
3   ("절마의 사랑을 듬뿍 받은 아이 그리고 절마니가 눈에 들어도 안 아파하는 아이였죠.", {"entities": []}),
4   ("절마니는 소리를 위해 귀여운 빨간 모자 하나를 만들어주었습니다.", {"entities": [{"16,24,"props"}]}),
5   ("그라서 소녀는 엄청 유명해졌어요, 그리고 모든 이들이 그녀를 빨간 모자를 쓴 아이로 부르게 되었어요.", {"entities": []}),
6   ("어느 날, 절마가 컷케이크를 몇 개 만들어주셔서 소녀에게 알렸어요.", {"entities": []}),
7   ("""매아, 절마가 컷케이크를 거부함, 아주 아프시다는 말을 들었지 뭐니, 절마니께 이 컷케이크 하나와 여기 이 베타 작은 병을 가져다 드리렴.""", {"entities": [{"47,54,"props"}, {"61,68,"props"}]}),
8   ("빨간 모자를 쓴 아이는 즉시 절마니를 보러 출발했어요.", {"entities": []}),
9   ("절마니는 다른 마을에 사셨거든요.", {"entities": [{"5,10,"location"}]}),
10  ("술을 통과하다 그녀는 늑대 영감을 만났어요.", {"entities": [{"0,1,"location"}]}),
11  ("늑대는 이 아이를 엄청 좋아하고 싶었어요. 하지만 감히 그러진 않았어요. 왜냐면 '씩씩한' 일꾼들이 숲 근처에 있었기 때문이죠.", {"entities": []}),
12  ("늑대가 그러보고 어디 가는 걸이라고 물었어요.", {"entities": []}),
13  ("황량한 아이는 나쁜 사람인 늑대와 대화를 나누며 가까이 하는 게 얼마나 위험한 일인지도 모르고 그에게 물었어요.", {"entities": []}),
14  ("""절마니 보러 가요, 이 컷케이크 하나와 베타 작은 병을 절마가 갖다드리라 하셨거든요.""", {"entities": [{"14,21,"props"}, {"23,30,"props"}]}),

```

〈예시〉 커스텀 NER 모델 생성을 위한 소셜 데이터 라벨링

1-2-2) 임베딩 및 라벨 예측: Tok2Vec 모델과 전이 기반 파서

Tok2Vec 모델은 텍스트를 벡터로 변환하는 역할을 하며, 이를 통해 Spacy 모델이 텍스트 데이터를 이해하고 처리할 수 있도록 한다.

전이 기반 파서(Transition-Based Parser)는 문장의 구조를 단계별로 분석하여 장소나 소품과 같은 개체명을 인식한다. 문장을 분석하면서 텍스트의 구조적 관계를 파악 후 각 단계에서 전이를 통해 문장의 특정 부분을 해석하여 개체명을 추출한다.

1-2-3) Spacy 모델 학습

모델 학습은 주어진 텍스트 데이터에 대해 적절한 라벨을 예측하는 과정이다. 이 과정에서 정답 라벨과 모델이 예측한 라벨 간의 차이를 계산하고, 이 차이를 줄이기 위해 경사하강법(gradient descent)을 활용하여 모델의 가중치를 조정했다. training data는 text와 label로 구성되어 있고, 학습시킨 Spacy 모델에서는 해당 text에 대해 label을 예측하도록 구현했다. 정답 라벨과 비교해서 도출된 차이만큼 gradient를 조정하면서 모델 개선을 진행했다.

1-3) 소설의 대사, 지문 분류 및 인덱싱

소설 텍스트에서 대사와 지문을 분류하는 작업을 진행한다. 문장 내 따옴표의 유무를 구별하는 정규표현식을 사용하였으며, 이 작업에서 각 문장에 인덱스를 표시하여 객체 배열로 반환한다.

```
{ 'type': 0, 'index': 2, 'sentence': '뭐! 이 고약한 것.' }  
{ 'type': 1, 'index': 3, 'sentence': '라며 여자마법사가 소리쳤어요.' }  
{ 'type': 0, 'index': 4, 'sentence': '뭔가 어찌고 어째! 너를 세상과 완전히 단절시켰는데도 이' }  
{ 'type': 1, 'index': 5, 'sentence': '화가 난 여자마법사가 라퐁젤의 아름다운 머리카락을 부여잡' }  
{ 'type': 1, 'index': 6, 'sentence': '그래서 그녀의 사랑스럽던 머리카락들이 바닥에 툭툭 떨어' }  
{ 'type': 1, 'index': 7, 'sentence': '그럼에도 불구하고 여전히 화가 안 풀린 이 무자비한 여자' }  
{ 'type': 1, 'index': 8, 'sentence': '거기서 라퐁젤은 큰 슬픔과 고통 속에 살아야했습니다.' }  
{ 'type': 1, 'index': 9, 'sentence': '하지만, 라퐁젤을 내뱉아 가둔 바로 그 날 저녁 여자마법' }  
{ 'type': 1, 'index': 10, 'sentence': '곤 왕자님이 나타나 소리쳤어요.' }  
{ 'type': 0, 'index': 11, 'sentence': '라퐁젤아, 라퐁젤아, 네 머리카락을 아래로 내려주렴.' }  
{ 'type': 1, 'index': 12, 'sentence': '여자마법사가 창에 매단 그 머리카락을 내려주었어요.' }  
{ 'type': 1, 'index': 13, 'sentence': '왕자님이 올라왔어요.' }  
{ 'type': 1, 'index': 14, 'sentence': '하지만 아름다운 라퐁젤은 온데간데없고 웬 여자마법사가' }  
{ 'type': 1, 'index': 15, 'sentence': '뭐예요.' }  
{ 'type': 0, 'index': 16, 'sentence': '아하!' }  
{ 'type': 1, 'index': 17, 'sentence': '라며 여자마법사가 조롱하듯 큰소리로 말했어요.' }  
{ 'type': 0, 'index': 18, 'sentence': '연인을 찾고 있던, 하지만 아름다운 새는 더는 동지에서' }
```

<예시> 소설 데이터의 형태

예시에서 객체 배열은 type, index, sentence에 대한 정보가 담겨있다. type이 0인 경우는 대사, type이 1인 경우는 지문에 해당한다. index는 문장의 순서를 나타내고 sentence에 문장이 담겨있다.

1-4) 발화자와 대사 매칭 알고리즘

```
Processing quote: '엘리사, 이곳이 이곳가 있던 장소야?'  
Candidate: '엘리사, Found in: 엘리사의 눈은 어딘가 멀리, 사라진 기억을 더듬고 있는 듯 보였다.', Distance: 4  
Candidate: '엘리사, Found in: 엘리사는 조용히 고개를 끄덕였다.', Distance: 10  
Candidate: '엘리사, Found in: 엘리사가 문을 열러 갈 때, 안에서 보고 자거운 목소리가 들렸다.', Distance: 13  
Candidate: '엘리사, Found in: 토마스의 얼굴은 어둠에 가려져 있었지만, 토마스가 누구인지 엘리사는 단번에 알아챘다.', Distance: 17  
Candidate: '엘리사, Found in: 오래된 엘리자, 엘리사. 하지만 기억하겠지. 너와 내가 마지막으로 만난 그날을.', Distance: 20  
Candidate: '엘리사, Found in: 엘리사는 주먹을 쥔 뒤였다.', Distance: 23  
Candidate: '엘리사, Found in: 우린 여기서 무엇을 해야 하는 거지, 엘리사?', Distance: 26  
Nearest mention for candidate '엘리사': '엘리사의 눈은 어딘가 멀리, 사라진 기억을 더듬고 있는 듯 보였다. (Distance: 4)  
Candidate: '제임스, Found in: 어두운 골목을 빠져나와, 제임스는 잠시 멈춰 섰다.', Distance: 3  
Candidate: '제임스, Found in: 불필요하게 느껴지는 도시의 공기가 제임스를 감쌌다.', Distance: 2  
Candidate: '제임스, Found in: 제임스는 옆에 서 있는 여인에게 조용히 물었다.', Distance: 1  
Candidate: '제임스, Found in: 그래, 제임스. 여자가 바로 이곳 도시야.', Distance: 3  
Candidate: '제임스, Found in: 제임스가 말했다.', Distance: 9  
Candidate: '제임스, Found in: 찾는 건 제임스 통해서만 가능할 거야.', Distance: 14  
Candidate: '제임스, Found in: 제임스는 경계하듯 소리의 출처를 찾았다.', Distance: 15  
Candidate: '제임스, Found in: 제임스는 두 사람 사이에 흐르는 긴장감에 한 발 물러섰다.', Distance: 24  
Candidate: '제임스, Found in: 제임스가 조심스럽게 물었다.', Distance: 27  
Nearest mention for candidate '제임스': '제임스는 옆에 서 있는 여인에게 조용히 물었다. (Distance: 1)  
Candidate: '토마스, Found in: 토마스의 얼굴은 어둠에 가려져 있었지만, 토마스가 누구인지 엘리사는 단번에 알아챘다.', Distance: 17  
Candidate: '토마스, Found in: 토마스가 여기에 왜...?', Distance: 18  
Candidate: '토마스, Found in: 토마스는 웃음을 지으며 한 걸음 앞으로 다가왔다.', Distance: 19  
Candidate: '토마스, Found in: 토마스가 노력하며 말했다.', Distance: 21  
Candidate: '토마스, Found in: "지금은 토마스를 상대해야 해.", Distance: 28  
Candidate: '토마스, Found in: 토마스가 들고 있는 담을 알고 있어.', Distance: 29  
Nearest mention for candidate '토마스': '토마스의 얼굴은 어둠에 가려져 있었지만, 토마스가 누구인지 엘리사는 단번에 알아챘다. (Distance: 17)  
Best candidate for quote '엘리사, 이곳이 이곳가 있던 장소야?': '제임스'
```

<예시> 발화자-대사 매칭 결과

대사의 발화자를 추론하고 대사와 매칭한다. CSS는 특정 발화문의 발화자 이름은 발화문 주변에서 1회 이상 언급될 것이라고 가정하며, 발화문마다 주변을 스캔하여 발화자 후보군을 구하고 후보별로 점수를 계산해 최종 발화자를 특징하는 방법이다. 전체 문서에서 CSS는 발화자 후보의 이름이 언급된 문장과 인스턴스의 중심에 존재하는 발화문을 포함하여, 둘 사이에 존재하는 문장들로 구성된다. 예를 들어 한 인스턴스에 두 명의 등장인물 이름이 언급될 경우, 이로부터 두 개의 CSS를 얻는다. 만약 특정 인물의 이름이 인스턴스에서 2회 이상 등장한다면, 즉 한 후보에 대하여 2개 이상의 CSS가 만들어진다면 발화문으로부터 가까운 거리(글자 수 기준)에 위치한 후보가 선택된다. 한 인스턴스 내에서 CSS는 언급되는 등장인물의 수만큼 만들어진다. 최종적으로 RoBERTa 모델에서 인용문

별로 발화자 후보별 점수가 나오게 되고, 인용문의 발화자일 확률이 가장 높은 후보자를 매칭하여 스크립트 형식으로 반환한다.

1-5) 비대사 지문으로부터 행위 지문 생성

등장인물의 행위(액션)를 디렉팅하기 위해 비대사 지문으로부터 행위 지문을 추출한다. 한국 형태소 품사 태깅(POS Tagging)으로 문장을 형태소 단위로 쪼개고 품사를 지정한 뒤, ①행위 주체가 드러나는 문장을 찾고 동사를 구분한다. 구분된 동사들 중에서도 인물의 행위만을 추출하기 위해 ②ETRI 언어분석 API를 사용하여 동사구가 주체를 나타내는 형태소를 모두 포함하는지 확인한다.

```
(늑대 영감을 만났어요)
(그녀보고 어디 가는 길이라고 물었어요)
(가능한 한 빨리 달리기 시작해 가장 빠른 지름길을 택했어요)
(나무열매들을 줍기도 하고, 나뭇들을 훑기도 하고, 만나는 귀여운 꽃들로 꽃다발을 만드느라 정신이 팔려 가장 먼 길을 택해 나아갔어요)
(편을 끌어당기니 정말 문이 열렸어요)
(문을 열고 할머니 침대 속으로 들어가 빨간 오자를 쓴 아이를 기다렸어요)
(우선 화들짝 겁이 났어요)
(가능한 한 가장 부드러운 목소리로 그녀에게 소리쳤어요)
(편을 끌어당기니 문이 열렸어요)
(웃을 빛고 침대 속으로 들어갔어요)

Process finished with exit code 0
```

<예시> 행위지문 추출

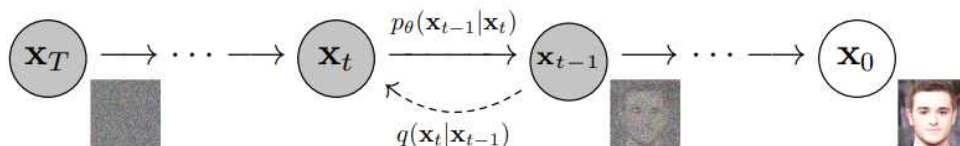
1-6) 대본의 형식으로 변환

앞서 1-1) ~ 1-5) 과정에서 추출한 값들을 기반으로 대본 형식으로 변환한다. 장소를 기준으로 씬(scene) 단위로 나누고, 인물의 액션을 기준으로 컷(Cut) 단위로 나눈다. 변환된 대본은 텍스트 파일(.txt)로 저장할 수 있다.

2) LoRA Stable Diffusion Model(LDM)로 이미지 생성, 스토리보드 생성

text-to-image 모델인 Diffusion Model을 사용하여 컷(Cut)으로 나눈 대본 텍스트 중 등장인물의 액션을 모델의 입력값으로 넣어 이미지를 생성하고, 대본 변환 과정에서 수집한 장소, 소품, 대사, 지문 등의 데이터를 조합하여 스토리보드를 생성한다.

2-1) 디퓨전 모델(Diffusion Model)



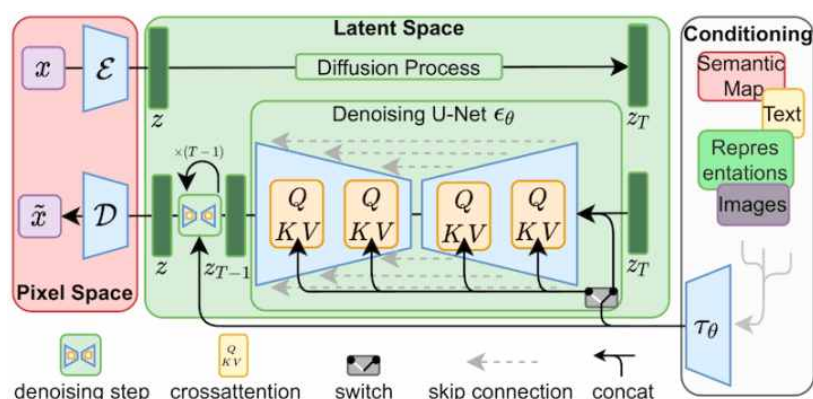
<그림> Diffusion Model

디퓨전 모델(Diffusion Model)은 생성형 모델 중 하나로, 원본 데이터로부터 잡음을 추가하는 잡음 데이터를 만드는 순방향 확산(forward diffusion), 완전한 잡음 데이터로부터 원본 데이터를 복원하는 역방향 확산(backward diffusion)을

거친다. 디퓨전 모델은 잡음 예측기(Noise predictor)를 통해 잡음이 추가된 데이터에 얼마나 많은 잡음이 들었는지 예측하고 이미지를 복원하는 역방향 확산 프로세스가 사용된다. 모델은 학습한 이미지의 스타일과 비슷한 새로운 데이터를 생성한다.

2-2) LoRA(Low-Rank Adaptation) Stable Diffusion Model

LoRA(Low-Rank Adaptation)는 디퓨전 모델을 학습시키기 위한 한 방법이다. 전체 가중치를 미세조정하는 대신, 전체 행렬을 랭크(space)가 낮은 두 개의 행렬로 분해하여 일부 가중치만 업데이트한다. 가중치는 프롬프트와 이미지가 교차하는 교차 인지 레이어(Crossattention)에서 조정된다.



<그림> LoRA Stable Diffusion Model

LoRA Stable Diffusion Model은 text-to-image로, 텍스트 프롬프트를 입력하면 그에 맞는 이미지를 생성한다. 사전 학습된 디퓨전 모델의 가중치에 새로운 학습을 위한 가중치를 추가 또는 업데이트하는 방식으로 모델을 학습한다. 실제 학습시켜야 하는 가중치의 수가 줄어들어 전체 가중치를 미세조정하는 방식보다 더 적은 양의 데이터를 사용한다. 그렇기에 빠른 속도로 가중치 학습이 가능하며 경량화된 모델을 얻을 수 있다.

본 서비스에서는 Civital에서 제공하는 Storyboard Sketch Safetensor와 Base Model인 SDXL 1.0을 사용하여 text-to-image 과정을 거쳐 이미지를 생성한다.

3) GPT-3.5을 활용한 소설 기획 과정에 가이드를 제공하는 챗봇 개발

OpenAI의 GPT-3.5-turbo-instruction을 활용하여, 사용자가 창작물을 기획하는 과정에서 도움을 주는 질의응답 챗봇을 개발한다. 이 모델은 지시를 따르는 데 특화되어 사용자의 질문을 이해하고 적절한 답변을 제공하는 데 강점을 가진다. 특히 대화가 길어지더라도 이전의 대화를 기억하여 일관된 답변으로 응답한다. 챗봇은 사용자가 작성한 소설을 기반으로 답변을 제공하며, 사용자의 질문에 맞춰 명확하고 간결하게 응답을 하도록 톤을 설정하여 튜닝했다.

4) React, Spring Boot를 사용한 웹서비스 구축 및 배포

React와 Spring Boot를 사용하여 웹서비스를 개발하고, 클라이언트와 서버 간 통신 환경을 구축한다. 클라이언트 단은 React-query와 axios를 통해 API를 호출하여 HTTP 요청을 진행한다. 서버 단에서 HTTP 요청을 받으면 클라이언트 단에 요청에 대한 응답과 데이터를 전송한다. 서버는 AWS EC2를 사용해 안정적인 가상 클라우드 컴퓨팅 서버에 배포한다.

4-1) React 서버 구축 및 배포

- 데이터 관리: react-query, context API

서버 상태의 데이터를 관리하기 위해 react-query를 사용하여 동일한 동작에 대한 API의 중복 호출을 방지하고 데이터를 최신 상태로 업데이트한다. context API를 사용하여 데이터를 전역으로 선언함으로써, 여러 컴포넌트에서 사용되는 데이터들을 하위 컴포넌트로 전달하여 발생하는 props drilling을 방지하고 코드의 품질을 높일 수 있다.

- 서버 배포 방법: vercel

웹 서비스는 vercel을 사용하여 배포한다. vercel은 웹사이트 배포를 할 수 있는 클라우드 플랫폼으로, 개발자가 즉각적인 배포 및 호스팅을 지원하며 웹페이지 보안을 위한 SSL 인증서와 HTTPS를 처리해준다.

4-2) Spring boot 서버 구축 및 배포

- 데이터 관리: MongoDB, Redis

데이터의 빠른 읽기/쓰기 작업에 최적화되어 있어 실시간 채팅 애플리케이션의 성능 요구사항에 맞춘 MongoDB를 사용했으며, Redis의 TTL 기능을 활용하면 저장된 토큰이 일정 시간이 지나면 자동으로 삭제되므로, 만료된 토큰을 관리할 필요 없이 인증 정보를 효율적으로 관리할 수 있다.

- 서버 배포 방법: 롤링(Rolling) 배포

롤링 배포는 서버 인스턴스를 순차적으로 업데이트하면서 기존 인스턴스가 서비스 중단 없이 계속 트래픽을 처리할 수 있게 하여 사용자에게 중단 없는 서비스를 제공할 수 있다. 또한, 새 버전 배포 중 문제가 발생할 경우, 문제 발생 범위를 최소화할 수 있다. 롤링 배포에서는 업데이트된 인스턴스에만 새로운 코드가 적용되기 때문에 전체 서비스에 영향을 주지 않고 문제를 해결할 수 있다. 마지막으로, 전체 서버를 한꺼번에 업데이트하지 않으므로 필요한 만큼의 리소스만 추가로 사용하게 되어, 클라우드 환경에서 비용 효율적인 배포가 가능하다.

4-3) Flask 서버 구축 및 배포

- 데이터 관리

output 이미지 스토리지로 AWS S3 버킷을 두어 사용하였고, 시스템 아키텍처에 따라 Flask server는 데이터베이스 모델에 직접적으로 접근하는 동작을 포함하지 않고, 모든 output은 Spring 서버로 바로 전달하여 처리하였다.

- 서버 배포 방법: Vast.ai GPU 서버 대여

GPU 서버 대여 플랫폼인 Vast.ai를 통해 GPU 인스턴스를 대여하였다. 다양한 GPU 환경 중, 비용을 최소화하면서도 11GB의 VRAM이 필요한 SDXL 모델을 원활하게 실행할 수 있는 Nvidia RTX 4060Ti 인스턴스를 선택하였다.

결과적으로 유사한 성능을 제공하는 AWS EC2 인스턴스에 비해 100배 가까이 저렴하게 사용할 수 있었다. (EC2 g4 시간당 1\$ 이상, Vast.ai: 시간당 0.08\$ + 메모리 유지비용 하루당 0.2\$) 간혹 메모리를 초과하는 등의 비상 상황에 대비하기 위해 추가로 한 대의 서버를 Cold Standby 상태로 준비해 두었다. Vast.ai에서는 인스턴스 간 내용 copy를 지원하여 두 서버가 동일한 환경을 유지하도록 할 수 있었다.

4. 프로젝트 개발 환경

구분		상세내용
S/W 개발환경	OS	Windows 11
	개발환경(IDE)	Visual Studio Code, IntelliJ, PyCharm
	개발도구	React, Spring Boot, Flask
	개발언어	Typescript, Java, Python
	DB	Postgresql, MongoDB
	기타사항	AWS EC2, AWS RDS
H/W 구성장비	디바이스	-
	센서	-
	통신	-
	언어	-
	기타사항	-
프로젝트 관리환경	형상관리	GitHub
	의사소통관리	Discord, KakaoTalk
	문서관리	Notion, Google Drive

5. 기타 사항 [본문에서 표현되지 못한 프로젝트의 가치(Value)] 및 제작 노력

- 자연어 처리 알고리즘 도출 과정의 창작성: 소설을 대본으로 변환하는 과정에서 필요한 정보를 추출하고 자연어 처리의 범위를 정하는 데 많은 고민이 필요했다. 이 과정은 프로젝트의 전반적인 성능을 결정짓는 핵심 요소였기에, 가장 큰 비중을 두었고 상당한 시간을 투자했다. 문단, 문장, 구, 단어, 형태소 단위로 텍스트를 분석하여 문장 성분을 분리하고 이를 재조립하는 방법을 연구하였다. 이러한 과정에서 단계별로 사용할 모델, 라이브러리, API 등의 레퍼런스를 확보하기 위해 광범위한 자료 탐색을 진행했다. 자연어 처리의 각 단계에서 문맥을 가장 정확히 이해하는 알고리즘을 도출하는 것을 목표로, 문제를 세부적으로 분해하고 순서를 변경하거나 재조립하는 다양한 시도를 하였다. 또한, 여러 모델을 도입하여 변환 결과의 정확도를 높이는 방법을 탐색했다.

1) 대본에서 주요 인물을 한정하여 대사를 주고받도록 하려면 등장인물이 명확하게 구분되어야 했다. 등장인물을 나타내는 여러 명사를 묶어내는 목표를 상호참조 해결(coreference resolution) 알고리즘을 통해 도달할 수 있을 것으로 보았다. 다양한 오픈소스 모델을 수집하여 비교한 결과, 정확도가 가장 높은 것으로 평가된 ETRI 상호참조해결 API를 선택하였다.

2) 영상물 촬영 시 설정해야 할 배경, 즉 무대와 소품을 준비하기 위해 소설 내의 장소와 소품을 추출해야 했다. 인물과는 구별되는 주요한 명사를 인식하기 위해, 검색 기법과 비슷하게 텍스트 내에서 특정 단어를 인식하는 NER(개체명 인식, Named Entity Recognition) 기법을 참고하여 소설 데이터 내에서 장소와 소품을 라벨링하여 장소 사전 데이터셋을 구축하고, 커스텀 NER 모델을 개발하였다.

3) 인물의 대사와 행위를 구분하여 대본에 표현하기 위해 대사와 비대사를 분리하였다. 소설 원작에서 인물의 발화로 표현되는 따옴표 속 대사는 그대로 사용하였다. 비대사 부분에서 화자의 독백을 분리하여 대사에 포함시키는 단계는 생략하였다. 큰따옴표, 작은따옴표, 괄호 등을 문장 내에 포함하는 경우를 제외하기 위해 정규표현식을 여러 단계로 작성하였다.

4) 자연어 처리 알고리즘을 순차적으로 거치는 동안, 글의 특정 부분을 전체 문장에 대해서 비교해야 하는 단계가 있었기에 문단 구분 없이 해체했던 자연어를 다음 단계에서 재조립할 필요가 있었다. 글의 선형적 흐름, 진행 순서를 보존하기 위해 문장 단위로 인덱스를 매겼다.

5) 발화자와 대사 매칭 알고리즘 발화자와 대사를 매칭하기 위해 CSN(Candidate Scoring Network) 모델을 사용하였다. 대사를 기준으로 발화자의 후보가 되는 인물을 추정할 수 있도록 세그먼트의 범위를 세밀하게 조정하였으며, 점수 산정 방법을 고민하였다.

6) 비대사에서 행위 지문을 추출하기 위해 품사 태깅, 의존 구문 분석, 의미역 인식 등 다양한 문장 분석 알고리즘을 적용하는 시도가 있었다. 단어 간의 의미 관계를 분석한 후, 행위의 주체가 되는 명사를 기준으로 인물의 행위를 나타내는 동사만을 구분해내기 위한 알고리즘을 추가로 도입하였다.

7) 대본을 스토리보드로 변환하기 위해 썬별로 구분되어 있던 데이터를 다시 컷 단위로 세분화하였다. 이미지를 묘사하는 프롬프트에는 인물의 동작이 묘사되어야 하므로 행위 지문을 기준으로 컷을 나누었고, 장소, 소품 등 자연어 추출 결과물을 덧붙여 재조립하였다.

- 구현 과정 중 일부 단계에서 사용할 라이브러리를 탐색할 때, 인공지능으로 처리해야 할 Task를 해결할 수 있는 API를 도입할 것인지, 같은 기능을 제공하는 오픈 소스 모델을 튜닝해서 사용할 것인지 검토해야 했다. 그 과정에서 모델을 비교를 위한 테스트 결과를 아래와 같이 정리하였다.

Aa 모델명	model type	epoch	batch size	learning rate	accuracy/loss	params
BERT NER_v1	bert-base-multilingual-cased	3		0.00002	0.89	ner_bert_model...
대본화 BERT_v1	klue/roberta-base	10	16	1e-5		csn.ckpt
대본화 BERT_v2	klue/roberta-base	10	16	1e-5		csn.ckpt
BERT NER_v2	bert-base-multilingual-cased	5		0.00002	0.89	ner_bert_model... ner_bert_model...
상호참조해결 모델	bert-base-multilingual-cased	3000		3e-5	0.2684/4.94	
spacy NER 모델_v1	spacy ko_core_news_sm	1600		5e-5		model-best.zip model-last.zip

보다 깊이 있는 연구를 위해, 심층적인 내부 구조를 공부하여 우리 프로젝트만의 모델을 구축하고자 소스코드를 뜯어보며 작업하는 방식을 택하려 했지만, 결국 모델의 성능이 기대에 미치지 못해 API로 대체한 부분이 있다. 대형연구시설 ETRI에서 공개한 API인 만큼 속도와 정확성 등의 성능은 보장되어 있었으나 이 또한 우리 프로젝트에 맞게 수정할 필요가 있었다. 다만 외부 API를 사용하면 우리 프로젝트에 맞는 추가 데이터를 학습시키지는 못한다는 단점이 있어, 우선 API의 출력을 신뢰하고 이를 후처리하는 방법을 추가로 고안하였다.

- 대본 서비스의 단계별 산출물인 대본과 스토리보드의 형식을 정할 때, 여러 형태를 검토하여 영상물 구성에 필수적인 요소를 선정하여 범용적으로 이용될 수 있는 양식으로 결정하였다. 스토리보드에는 해당 썬 촬영장소와 썬 내용 요약

표시, 컷 이미지의 비중을 크게 하고, 컷별로 행동과 액션을 나타내었다.

- 소설이 단계를 거둡니다. 변환되는 과정을 알기 쉽게끔 사용자 인터페이스를 선형적인 흐름으로 구성하였다. 왼쪽에서 오른쪽으로 진행하면서 다음 단계의 컴포넌트가 점점 생겨나는 가로로 긴 레이아웃으로 표현하였다. 시간이 소요되는 작업은 대기 중임을 알 수 있도록 스피너를 표시하였다.



III. 프로젝트 수행 내용

1. 프로젝트 수행 일정

프로젝트 기간 (ICT멘토링 사이트 기준)		2024.04.01. ~ 2024.10.31.							
구분	추진내용	프로젝트 기간							
		3월	4월	5월	6월	7월	8월	9월	10월
계획	프로젝트 주제 선정, 구성 기능 구체화								
	각 기능별 적용 기술 선정								
	서비스의 필요성 및 활용성 검토								
분석	사용자 및 시스템 요구사항 분석								
	화면 시나리오 분석								
	팀원 역할별 기술 습득								
설계	UI/UX 세부 요소 설계								
	SW 아키텍처 설계								
	프로토타입 동작 과정 설계								
개발	데이터 수집 및 전처리								
	대본화 알고리즘 설계 및 구현 (NLP)								
	소셜 정보를 기반으로 챗봇 학습 및 구현								
	스토리보드 생성 기능 구현(text to image)								
	백엔드 API 서버 개발								
	웹페이지 UI 개발								
테스트	알파테스트, 베타테스트 수행								
	도메인 관련 종사자 피드백 수집, 기능 보충								
종료	개발 보고서 작성								
	이브와 공모전 참가								

2. 프로젝트 추진 과정에서의 문제점 및 해결방안

1) 프로젝트 관리 측면

- 개발 진행 과정 문서화 및 트래킹: 주간 회의를 통한 업무 현황 보고만으로는 전체 작업규모를 파악하거나 개발 속도를 조절하는 부분에서 어려움이 있었다. 팀 내 PM의 의견으로 제품 백로그를 Notion에 문서화하였다. 서버별, 기능별로 작업 단위를 나누고, 하위 작업을 세분화하여 주차별 진행상황을 추적하기 쉽도록 했다.

- 컨벤션을 통한 일관된 스타일 유지

(1) 커밋 메시지 컨벤션: 각자가 깃허브 commit을 등록할 때 작성하는 메시지의 형식이 다르다면 타 팀원들이 commit에서 어떤 업무를 수행했는지 파악하는 데 어려움이 생길 것이라고 보았다. 이를 대비하여 GitHub 레포지토리 하위의 .github 폴더 내 템플릿 커스텀 기능을 이용해 커밋 컨벤션 양식을 만들었고, 커밋 컨벤션 양식을 적용할 수 있는 방법을 제시하여 팀원 모두가 동일한 커밋 메시지 형식으로 작성할 수 있도록 했다.

```
#[타입] 제목 임력 *M
#M
# 본문은 아래에 작성 *M
#M
# 코밋말은 아래에 작성: ex) Github issue #23 *M
#M
# --- COMMIT END --- *M
#M
# <타입> 리스트 *M
#M
# feat : 기능 (새로운 기능) *M
# fix : 버그 (버그 수정) *M
# refactor : 리팩토링 *M
# design : CSS 등 사용자 UI 디자인 변경 *M
# comment : 필요한 주석 추가 및 변경 *M
# style : 스타일 (코드 형식, 세미콜론 추가: 비즈니스 로직에 변경 없음) *M
# docs : 문서 수정 (문서 추가, 수정, 삭제, README) *M
# test : 테스트 (테스트 코드 추가, 수정, 삭제; 비즈니스 로직에 변경 없음) *M
# chore : 기타 변경사항 (빌드 스크립트 수정, assets, 패키지 매니저 등) *M
#M
# init : 초기 생성 *M
# rename : 파일 혹은 폴더명을 수정하거나 옮기는 작업만 한 경우 *M
# remove : 파일을 삭제하는 작업만 수행한 경우 *M
#M
# 제목은 명령문으로 *M
#M
# 제목 끝에 마침표(.) 금지 *M
# 제목과 본문을 한 줄 띄워 분리하기 *M
# 본문은 *어떻게* 보다 *무엇을*, *왜*를 설명한다. *M
# 본문에 여러줄의 메시지를 작성할 땐 "-"로 구분 *M
#M
# <코리얼> *M
```

[그림] 커밋 메시지 템플릿



[그림] 커밋 내역

(2) 코드 컨벤션: AI 서버를 모든 팀원이 함께 개발하는 과정에서 네이밍 방법이 나 조건문과 반복문을 작성하는 경우 코드 스타일이 통일되지 않는 문제가 있었다. 이를 해결하기 위해 코드 컨벤션을 작성하고 팀원들에게 숙지를 요청하였고 팀원들은 컨벤션에 따라 코드를 수정하였고, 코드의 가독성과 품질을 높일 수 있었다.

코딩 컨벤션

1. 코드 레이아웃

- 들여쓰기: 공백 4칸(tab 1번, tab 1번이 스페이스 바 4번인지 확인하기)
- 최상위(top-level) 함수와 클래스 정의는 2줄씩 띄어 쓴다.
- 클래스 내의 메소드 정의는 1줄씩 띄어 쓴다.
- 불필요한 공백 지양

```
# Wrong
spam( ham[ 1 ], { eggs: 2 } )

#Correct
spam(ham[1], {eggs: 2})

# Wrong
if x == 3 :
    return x , y

# Correct
if x == 3:
    return x, y
```

```
# Wrong
import pandas, numpy

# Correct
import pandas
import numpy
```

아래와 같이 사용해도 된다.

```
from numpy import square, sign
```

2) 조건

- None 을 비교할 때는 is 나 is not 만 사용한다.
- boolean 타입의 값을 조건문에서 == 를 통해 비교하지 않는다.

3) 문자열

- 문자열을 표현할 때 쌍따옴표("")로 통일한다.
- string 모듈보다는 string 메소드를 사용한다. 메소드는 모듈보다 더 빠르고, 유니코드 문자열에 대해 같은 API를 공유한다.

4) 예외

- 예외를 except: 로 잡기보단 명확히 예외를 명시한다.
e.g. except ImportError:
- try: 블록의 코드는 필요한 것만 최소한으로 작성한다.
- 클래스 기반의 예외를 사용한다.

```
class MessageError(Exception):
    """Base class for errors in the email package."""
```

[그림] 코딩 컨벤션 문서

2) 프로젝트 개발 측면

- **개발 자원에 따른 프로젝트 규모 재정비:** 서비스를 기획하는 과정에서 풍부한 사용자경험을 제공하기 위해 등장인물의 외형과 인상착의 추출, 유사한 배우 가상 캐스팅, 오디오북, 스토리보드 배경음악 추천 기능 등 다양한 아이디어가 논의되었다. 하지만 7개월 간 투입할 수 있는 개발 자원은 한정적이었으므로 이를 고려하여 초점을 둘 핵심 기능을 다시 선정하였고, 서비스 시나리오를 확정하며 프로젝트의 규모를 재정비하였다.
- **NLP 작업의 어려움:** 자연어 처리 모델 개발 경험이 적었기에 단계별로 사용할 모델과 api를 결정하는 과정에서 시행착오가 많았다. 기획한 기능에 대해서 레퍼런스 삼을 만한 유사 프로젝트가 없어 대본화 기능을 구현하는 과정에서 단계별 알고리즘을 일일이 고안하였다. 개발 회의를 거듭하며 각 문장을 직접 분석하고 특징을 찾아내 의존 구문 분석(Dependency Parsing), SRL(의미역 인식) 알고리즘, 상호참조해결(Coreference Resolution), CSN(참여자 점수화 네트워크)을 검토하였고, 품사 태깅(PoS Tagging)을 단계를 세분화하여 진행했으며 최종적으로 예상 결과물에 근사한 결과물을 만들어낼 수 있었다.
- **CSS 작업의 어려움:** 소설의 종류와 내용에 따라서 발화자와 발화문 사이에 다른 후보군 언급과 비대사의 양에 따라 성능 평가에 영향을 미치는 경우가 발생하였다. 2개 이상의 css가 만들어지면 발화문으로부터 가까운 거리(글자 수 기준)으로 적용하는 로직의 개선점을 발견하였다. 전체 소설에 등장인물 선정 작업을 전 단계로 추가하여 발화 빈도가 높은 등장인물에 높은 가중치를 부여해 정확도를 높일 수 있었다.

3. 프로젝트를 통해 배우거나 느낀 점

- 프로젝트 진행에 있어 아이디어와 알고리즘의 단계를 구체화하며 로직을 짜고 생소한 모델을 많이 다뤄볼 수 있는 기회였다. 단계적으로 나아가며 새로운 기능을 만들어 내고 무에서 유를 창조해 내는 것에 어려움이 많았지만 팀원들과 고민하며 소통과 협업의 중요성에 대해 깨달으며 성장해 나갈 수 있었다. 어려움을 겪을 때마다 방향을 제시해주시고 중심을 잡아주시는 멘토님과 팀장님께 감사드리며 남은 기간 잘 마무리해 팀원들과 좋은 프로젝트를 완성해 나가고 싶다.

- 프로젝트 초기에 어떻게 하면 프로젝트를 잘 이끌 수 있을지, 어떤 방식을 채택하면 나를 포함해 팀원들의 개발 능력을 100% 발휘할 수 있는 환경을 조성할 수 있을지에 대해 고민했다. 본 프로젝트는 AI 서버에서 도출되는 결과물에 따라 프로젝트의 퀄리티가 좌우되기 때문에 AI 서버를 먼저 구축하고 웹페이지를 구현하는 방향으로 실행한 케이스다. 단지 AI, 웹페이지의 개발 순서가 바뀌었을 뿐인데 자연어 처리에 대한 기본 지식이 부족해서인지 간단하게 갈 수 있는 길일지라도 멀리 돌아서 갔던 것 같다. AI 서버 개발 자체에 많은 시간을 소요한 점이 아쉬웠다. 특히 한국어는 어근, 접사에 의해 단어의 품사가 결정되는 교착어고, 띄어쓰기가 잘 지켜지지 않아 어절 단위로 쪼개어 자연어를 다루는 방식이 처리에 있어서 어려움이 있었다. 팀장인 내가 이러한 문제가 발생할 것을 예상하지 못하고 프로젝트의 볼륨을 파악하지 못했던 점이 시간 자원 낭비의 결과를 가져왔다고 생각한다. 그렇지만 부족한 지식은 다양한 시행착오를 낳았고, 그 과정에서 자연어 처리 분야의 지식을 배울 수 있어 좋은 시도, 유익한 경험이라고 생각한다.

또, 프로젝트에서 적용하길 잘했다고 생각하는 문화는 '코드 리뷰'이다. 나는 코드를 명확한 변수, 다른 사람들이 봐도 알아보기 쉬운 클린 코드로 작성하는 것을 목표로 하지만 숙지하고 있는 언어로 코드를 작성하다 보면, 내게 편하고 익숙한 함수, 속성을 사용한 코드의 형태로 작성하려는 경향이 있다. 나에게 코드 리뷰는 자기반성의 시간과도 같았다. 최근에 팀원의 코드를 리뷰하면서 React의 생명주기 중 언마운트 단계에 `textInput`의 `focus` 함수를 작동시키는 코드를 보았다. 이전까지 이론적인 내용이 실전에 주는 영향력에 대해 크게 느끼지 못했는데, 이 코드를 보고 이론의 중요성을 새삼 깨닫게 되었다. 서로의 성장을 도모할 수 있는 코드 리뷰 문화를 지속하기 위해 내 코드뿐만 아니라 팀원의 코드에도 관심을 가져야겠다고 다짐한다.

- 모델을 선택하는 단계에서부터 적합한 알고리즘을 찾고, 이를 실제 데이터에 어떻게 적용할지 고민하는 과정이 흥미로웠지만 동시에 여러 어려움에 직면하면서 앞으로의 성장에 중요한 밑거름이 된 것 같은 생각이 들었다. 발화자 매칭을 위한 자체 알고리즘을 개발하는 과정에서는 창의성과 기술적 역량이 동시에 요구

되었다. 기존의 알고리즘을 참고하면서도, 우리의 프로젝트에 맞는 독창적인 접근법을 찾아내는 것이 중요했다. 많이 사용되는 음성 기반이 아닌 텍스트 특성을 고려한 알고리즘 설계가 큰 도전이었다. 문장 구조의 특징을 포착하고, 이를 기반으로 발화자를 매칭하는 알고리즘을 설계로 여러 번의 실패와 수정 과정을 거치면서, 점점 더 개선된 알고리즘을 개발할 수 있었다. 스프링부트에서 비동기 프로그래밍을 활용하여 병렬 처리를 가능하게 시도했다. 이는 특히 여러 외부 API와의 통신이 필요한 상황에서 비동기 처리를 통해 요청이 동시에 이루어지도록 하고, 결과를 기다리지 않고 다른 작업을 수행함으로써 전체 처리 속도를 크게 개선할 수 있었다. 이로 인해 사용자 경험이 향상되었으며, 시스템의 응답성이 더욱 원활해졌다. 속도 개선은 단순히 기술적 도전 과제가 아닌, 실제 사용자에게 더욱 빠르고 원활한 서비스를 제공하는 데 필수적인 요소임을 깨닫게 되었다.

IV. 기대효과 및 활용분야

1. 프로젝트의 기대효과

1. 작품의 기대효과

- 소설을 기반으로 극본, 스토리보드를 거쳐 연극 또는 영화 등의 영상물로 발전시키는 과정은 다양한 매체로의 IP 확장과 콘텐츠 융합을 꾀할 수 있다. 이는 문화의 향유층인 독자로 하여금 실감나는 경험을 가능하게 하며, 새로운 관객층에게도 접근할 기회를 제공한다.
- 본 서비스는 텍스트를 영상물의 형태로 변환하는 과정에서 소설의 극본화와 스토리보드 제작을 자동화하여 시간과 자원을 효율적으로 절감하도록 돕고, 창작물의 다양한 재해석을 예상 가능한 형태로 제시한다.
- 소설의 내용을 학습한 챗봇을 통해 향상된 검색 기능을 제공한다. 영상물로의 전환 과정에 필요한 세부적인 정보를 곧바로 질의하여 탐색 시간을 단축할 수 있다. 또한 작품 내 설정과의 충돌을 방지하여 창작 과정에서의 일관성 유지를 돕고, 2차 창작물 제작 시에도 참고 자료로 활용할 수 있다.

2. 참여 멘티의 교육적 기대효과

- 자연어 처리(NLP), 멀티 모달 AI(Text-Vision), 미디어 변환 과정 학습을 통해 해당 분야의 전문 지식을 쌓을 수 있다. 이 과정에서 본 서비스에 특화된 인공지능 모델의 성능을 평가하고 향상시키기 위해 심화 학습과 성능 개선 과정을 진행한다.
- 자연어 처리 과정에서 설명문 등의 형식에 구애받지 않고 제작된 창작물인 소설을 자료로 활용하고, 단순 요약과 분류를 넘어 재배열, 재구성하는 알고리즘을 설계함으로써, 고난도의 구조화 방법을 탐구해볼 수 있는 기회가 주어진다.
- 생성형 AI의 산출물을 단계적으로 활용하는 기능을 개발함으로써 최신 기술과 융합된 관점을 프로젝트에 적용해볼 수 있다.

2. 프로젝트의 활용분야

- 대본과 스토리보드 생성을 자동화함으로써 영화 및 TV 드라마 제작 과정의 초기 단계에서 투입해야 하는 시간과 비용을 절감하는 효과가 있다. 감독과 아티스트 등 영상물 제작 인력으로 하여금 더욱 빠르고 명확하게 시각적 상상을 구체화하도록 돕는다. 이로써 전체 제작 일정을 단축시켜 창의적인 작업에 더 많은 시간과 자원을 투자할 수 있다.

- 비주얼 노벨 형태의 게임 개발에도 활용될 수 있다. 게임의 스토리와 캐릭터 대화를 자동으로 생성하여 게임의 내러티브와 시각적 요소를 효율적으로 구성하는데 기여할 수 있다. 게임 씬을 시각화한 스토리보드를 생성함으로써 빠르게 프로토타입을 만들고, 스토리라인 테스트와 피드백을 효율적으로 수행할 수 있다.
- 글 콘티를 영상물로 변환하는 차원에서 광고 캠페인, 프로모션 비디오, 기업 프레젠테이션 등 마케팅 콘텐츠 제작에도 도입할 수 있다.
- 다양한 매체로 IP를 확장할 수 있을 뿐만 아니라, 저학년 교실에서 모둠 활동을 위한 연극 수업에 활용 및 어린이 교육자료로 개발하여 상업적 매체 전환 외에도 교육적 가치를 높이는 서비스가 될 수 있다.
- 한국 고전문학과 전래동화를 활용하여 국극 형태의 극본을 개발함으로써, 전통적인 이야기를 현대적인 무대에서 재해석할 수 있다. 우리 문화를 홍보하는 문화 콘텐츠로 활용하여 문화산업의 확장에 기여할 수 있다.